

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 06 -

CI/CD E AUTOMAÇÃO DE PIPELINES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	20
O QUE VOCÊ VIU NESTA AULA?	24
REFERÊNCIAS.....	26

EMSE

O QUE VEM POR AÍ?

Na aula passada vimos que, após colocar um modelo de Machine Learning em produção, o trabalho está longe de terminado – modelos de ML não são sistemas estáticos e seu desempenho tende a degradar com o tempo devido a mudanças de dados e do ambiente. Para enfrentar esse desafio, esta aula introduz práticas de CI/CD aplicadas a Machine Learning, no contexto de MLOps. Vamos explorar técnicas de integração e entrega contínua, mostrando como pipelines automatizados podem tornar o ciclo de vida de modelos mais confiável, reproduzível e ágil.

Nesta aula, você aprenderá: conceitos sobre CI/CD em projetos de Machine Learning, entendendo sua definição, benefícios e desafios; integrar ferramentas de versionamento de código e artefatos, testes automatizados e automação de deploy de modelos; explorar ferramentas populares como GitHub Actions, Jenkins e GitLab para orquestração de pipelines, além do uso de frameworks como MLFlow (para rastreamento e registro de modelos) e Kubeflow (para pipelines de ML em escala); compreender conceitos de Infraestrutura como Código (IaC) e containerização no contexto de MLOps; e adotar práticas de engenharia de software em ML, incluindo validação automatizada de dados, testes de modelo e ambientes reproduzíveis.

Também teremos um hands on passo a passo, configurando um pipeline CI/CD usando GitHub Actions integrado ao MLFlow – onde um simples commit no repositório aciona testes automatizados e, se tudo estiver correto, registra/atualiza um modelo no MLFlow com total rastreabilidade dos experimentos. Você verá exemplos de código (YAML e Python) ilustrando essa automação.

Em resumo, esta aula mostrará como aplicar os princípios de DevOps no mundo de ML – permitindo integrar novos códigos e dados de forma contínua, testar modelos automaticamente e entregar atualizações de forma confiável no ambiente de produção, fechando o ciclo de vida do modelo com manutenção contínua. Preparado? Então vamos criar nosso primeiro pipeline de CI/CD para Machine Learning.

HANDS ON

Para entender como implementar um pipeline CI/CD de Machine Learning na prática vamos propor um cenário. Imagine o seguinte: você possui um projeto de ML em um repositório GitHub – por exemplo, nosso modelo de previsão de doenças cardíacas treinado com Python – e deseja automatizar o processo de teste e implantação desse modelo sempre que fizer alterações no código. Além disso, quer registrar os resultados (metrics, modelo) em uma ferramenta de tracking (MLFlow) para manter a rastreabilidade.

O objetivo é configurar uma workflow de GitHub Actions que, a cada push no branch principal, execute os seguintes passos: (1) instalar dependências e rodar testes no código; (2) treinar (ou retreinar) o modelo; (3) opcionalmente, se o modelo for satisfatório, registrar a nova versão no Model Registry do MLFlow ou implantar o modelo; (4) avaliar o pipeline e reportar os resultados; tudo isso de forma automática.

Vamos acompanhar mais detalhadamente na seção Saiba Mais.

SAIBA MAIS

Considere que nosso repositório tem a seguinte estrutura básica, seguindo o que aprendemos nas aulas anteriores:

heart-disease-prediction/

- |— data/ (dados de treino/validação)
- |— src/
 - | |— train.py (treinamento do modelo)
 - | |— register.py (registro do modelo)
 - | |— utils.py (funções auxiliares)
 - | |— tests.py (testes unitários)
- |— mlflow_runs/ (diretório onde salvar experimentos do mlflow)
- |— requirements.txt (dependências Python)
- |— README.md

Vamos imaginar que train.py lê os dados e treina o modelo; na sequência, o tests.py poderia executar alguns testes unitários que checam, por exemplo, se as funções em train.py estão funcionando adequadamente e o register.py registraria os artefatos gerados no MLflow caso o novo modelo treinado fosse melhor que o anterior já registrado. Em outras palavras, garantiríamos que o novo modelo treinado só fosse para “produção” caso tivesse métricas (acurácia, F1-Score, etc.) melhores que o modelo já em produção.

Para atingir este objetivo, podemos criar pipelines de CI/CD (Integração Contínua e Entrega Contínua) automatizados que garantam que o código seja testado e implantado de forma eficiente e confiável antes de colocarmos em produção. Utilizaremos para isso o conceito de Workflows no GitHub Actions. No GitHub, criaremos o arquivo de workflow em .github/workflows/ci_ml_pipeline.yml. Esse YAML descreve os jobs que o Actions irá executar. A figura 1 traz um exemplo simplificado do conteúdo.

```
github > workflows > % cd_ml_pipeline.yml
1  name: CI Pipeline ML
2
3  on:
4    push:
5      branches: [ "main" ]
6
7  jobs:
8    build_and_train:
9      runs-on: ubuntu-latest
10     steps:
11       - name: Checar código
12         uses: actions/checkout@v3
13
14       - name: Configurar Python
15         uses: actions/setup-python@v3
16         with:
17           python-version: '3.9'
18
19       - name: Instalar dependências
20         run: pip install -r requirements.txt
21
22       - name: Executar testes
23         run: pytest tests/ --maxfail=1 -q
24
25       - name: Treinar modelo e logar no MLflow
26         env:
27           MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
28         run: python src/train.py
```

Figura 1 - Exemplo simplificado de pipeline no GitHub Actions
Fonte: Elaborado pelo autor (2026)

Vamos entender o que fizemos nesse workflow:

- Definimos que o pipeline será acionado em evento de push na branch "main".
- O job `build_and_train` roda em um runner Ubuntu. Nos steps:
 - Usamos a ação oficial `actions/checkout@v3` para obter o código do repositório.
 - Configuramos o Python 3.9 no runner.
 - Instalamos as dependências do projeto.
 - Executamos os testes (com PyTest neste exemplo). Se algum teste falhar, o job para aqui (o Actions marca falha).
 - Se os testes passarem, partimos para treinar o modelo. Note o uso de `env`: para definir variáveis de ambiente – estamos pegando credenciais do MLflow do repositório (armazenadas de forma segura em Settings > Secrets do GitHub). As variáveis `MLFLOW_TRACKING_URI` e outras podem ser usadas dentro do `train.py` para conectar ao servidor do MLflow.
 - Ex.: poderíamos ter no `train.py` algo como `mlflow.set_tracking_uri(os.environ["MLFLOW_TRACKING_URI"])`

) para definir que os logs vão para um servidor remoto, ou deixar vazio para usar local.

Esse pipeline básico garante que, a cada mudança, nosso código é testado e o modelo é treinado com rastreabilidade. Os artifacts gerados poderiam também ser guardados como artefato do Actions usando o step actions/upload-artifact, se quisermos baixar o modelo resultante diretamente do CI.

No exemplo anterior, usamos variáveis de ambiente para conectar ao MLFlow. Isso pressupõe que temos um MLflow Tracking Server rodando (pode ser um EC2, um workspace do Databricks etc.). Outra abordagem, se não tivermos um server, é usar MLFlow em modo local (logs em arquivo) – não requer credenciais, mas dificulta acesso compartilhado. Vamos supor que temos sim um MLFlow tracking server acessível.

Para efeitos didáticos, imagine que o src/train.py contém algo como:

```
1 import mlflow
2 from mlflow import sklearn
3 from sklearn.datasets import load_iris
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7
8 # Conectar ao tracking server (URL definida em MLFLOW_TRACKING_URI)
9 mlflow.set_experiment("MLPipeline_Exemplo") # garante que estamos em um experimento nomeado
10
11 # Carregar dados e treinar modelo simples
12 X, y = load_iris(return_X_y=True)
13 X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
14
15 with mlflow.start_run():
16     params = {"n_estimators": 100, "random_state": 42}
17
18     model = RandomForestClassifier(
19         n_estimators=params["n_estimators"],
20         random_state=params["random_state"]
21     )
22     model.fit(X_train, y_train)
23
24     # Avaliar
25     preds = model.predict(X_test)
26     acc = accuracy_score(y_test, preds)
27
28     # Logar parâmetros e métricas no MLflow
29     mlflow.log_params(params)
30     mlflow.log_metric("accuracy", float(acc))
31     # Salvar e logar o modelo treinado
32     sklearn.log_model(model, artifact_path="model")
```

Figura 2 - Exemplo script de treino de modelo com log no MLFlow
Fonte: Elaborado pelo autor (2026)

Esse código, quando executado dentro do GitHub Actions, irá registrar tudo no MLFlow. No UI do MLFlow (ou via API) poderemos ver um novo Run criado dentro do experimento "MLPipeline_Exemplo", com a métrica de acurácia e o modelo armazenado. Esse modelo recebe um ID e pode ser promovido a Production no Model

Registry se assim desejarmos. O importante aqui é: conseguimos automação com rastreabilidade – cada commit acionou um run no MLFlow com resultados, então sabemos exatamente qual versão do código (commit hash) gerou qual modelo (graças à integração do MLflow podemos salvar também o Git commit como tag do run, há funções para isso).

No fim da execução do Actions, se tudo der certo, teremos: testes aprovados e novo modelo treinado logado. Poderíamos estender o workflow YAML para, por exemplo, notificar via Slack se o pipeline falhar ou tiver sucesso, usar ações de cache para acelerar a instalação de dependências etc., mas mantivemos simples para foco no essencial.

No exemplo anterior, incluímos CI e o registro do modelo (entrega contínua até um registro de modelos). O próximo passo seria implantar esse modelo em um ambiente de serving automaticamente. Poderíamos fazer isso de várias formas, citando duas comuns:

- **Via Model Registry + Webhooks:** o MLFlow Model Registry possui recurso de webhooks, isto é, quando um modelo muda de estágio, pode acionar uma URL. Poderíamos configurar um webhook que, quando um modelo é marcado como "Staging" ou "Production", chame um endpoint que inicia a implantação (por exemplo, um script de deploy ou um CI job específico). Assim, ao final do nosso pipeline, poderíamos adicionar um step que verifica a acurácia: se acima de um limiar, promove o modelo a Staging no MLFlow via API – isso acionaria o webhook e, por exemplo, dispararia um job de implantação no Kubernetes ou em um Workspace Databricks via Model Serving.
- **Via continuação do Workflow CI/CD:** alternativamente, podemos colocar passos adicionais no YAML para implantação. Por exemplo, após train.py, adicionar:

```
- name: Deploy model to API
  if: success() # só roda se steps anteriores sucederam
  run: python deploy.py
```

Figura 3 - Exemplo deployment task no pipeline do Github Actions
Fonte: Elaborado pelo autor (2026)

Nesse `deploy.py` teríamos código para usar o modelo recém logado (usando MLFlow client ou pegando o artifact gerado) e fazer o deploy. O deploy poderia ser disponibilizar o modelo para um storage acessível pelo serviço de inferência ou chamar alguma API de serviço (ex.: rodar um comando de deployment no Databricks, ou um `kubectl apply` para Kubernetes etc.). No geral, para segurança, muitas equipes optam por Entrega Contínua (entrega automatizada até um ambiente de teste) e Implantação Manual em Produção. Por exemplo, nosso Actions poderia implantar automaticamente em um ambiente de staging para teste interno, mas requerer aprovação para promover a produção. O GitHub Actions suporta manual triggers (jobs que aguardam aprovação). Isso combina automatização com controle humano quando necessário (especialmente importante em ML que pode ter implicações significativas se algo sair errado em produção).

Após configurar tudo, fazemos um commit no branch main alterando, por exemplo, um hiperparâmetro no código. O GitHub Actions será acionado. Podemos acompanhar no tab "Actions" do repositório: ele mostrará nosso workflow CI Pipeline ML em execução. Etapas de checkout, instalação etc. vão aparecendo com logs em tempo real. Se um teste falhar, veremos lá o erro (e precisaríamos corrigir o código e commitar de novo, garantindo que só modelos com código aprovado chegam adiante).

Quando chegar na etapa de treinamento, ela pode demorar alguns minutos dependendo do modelo. No final, o Actions marca o run como verde (sucesso) se tudo passou. Podemos então analisar cada um dos estágios e tarefas realizadas.

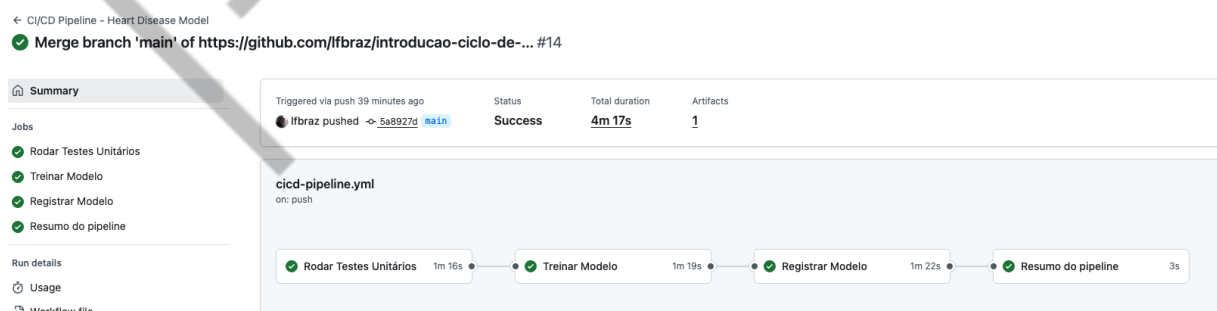


Figura 4 - Exemplo de execução de Pipeline de CI/CD no GitHub Actions
Fonte: Elaborado pelo autor (2026)

Em desenvolvimento de software tradicional, CI (Integração Contínua) significa mesclar alterações de código com frequência e testá-las automaticamente, enquanto CD pode referir-se a Entrega Contínua (preparar versões prontas para implantação) ou Implantação Contínua (implantar automaticamente em produção). Estas práticas, inicialmente pensadas para software, agora se tornaram cada vez mais relevantes também para ML, dando origem ao termo MLOps – uma abordagem que une DevOps e Data Science para facilitar a entrega confiável de modelos de ML.

No contexto de MLOps, a Integração Contínua ganha novos elementos: não apenas código, mas também dados e modelos passam a fazer parte da integração. Toda vez que o código ou os dados de treinamento são atualizados, idealmente um pipeline de ML deve ser reexecutado de forma automatizada, garantindo versionamento e reprodutibilidade dos resultados. Isto significa que podemos treinar e avaliar novamente o modelo a cada mudança, comparando com versões anteriores e detectando melhorias ou regressões de desempenho.

Pull requests no repositório podem disparar execuções de experimentos e até agendamentos periódicos podem ser usados para incorporar novos dados continuamente. Importante notar que testes automatizados continuam válidos aqui: além de testes de unidade no código (por exemplo, verificar se funções de pré-processamento retornam o esperado), podemos ter testes específicos de dados (ex.: checar se não há valores nulos inesperados) e até testes simples no modelo (ex.: verificar se um modelo treinado com um pequeno subset converge ou produz saída dentro de um intervalo esperado).

Já a Entrega/Implantação Contínua em ML envolve automatizar não só o deploy de aplicações, mas sim do modelo treinado em si. Ou seja, após um novo modelo ser treinado e validado automaticamente, ele pode ser entregue a um repositório de modelos ou até implantado em um serviço de inferência com o mínimo de intervenção humana. Isso torna o ciclo de realimentação mais rápido – usuários ou sistemas começam a usar as melhorias do modelo mais cedo – e permite incorporar feedbacks rapidamente em novas versões do modelo.

Em CD para ML é essencial incluir verificações como: garantir que o modelo é compatível com a infraestrutura de produção (dependências, versão de pacote, recursos de CPU/GPU disponíveis etc.), testar o serviço de predict com entradas conhecidas para ver se responde corretamente, medir a latência e throughput do

modelo sob carga e só então promover a versão para produção. Muitas vezes adota-se um fluxo de promoção de estágio, por exemplo: após passar nos testes de CI, o modelo pode ser automaticamente implantado em um ambiente de staging (teste) — isso seria uma Entrega Contínua.

A partir daí, talvez apenas mediante aprovação ou critérios de qualidade, ele é promovido manual ou semiautomaticamente à produção (quando falamos em Implantação Contínua, seria totalmente automático até prod, mas nem todas as empresas adotam deploy 100% automatizado para modelos por questões de controle). Resumindo, CI/CD em ML busca automatizar desde a integração de mudanças até a entrega de modelos treinados, garantindo qualidade em cada etapa.

Um ponto importante é que dados e modelos versionados passam a ser ativos cruciais no pipeline CI/CD de ML. Isso traz desafios adicionais em relação ao DevOps tradicional. Precisamos pensar em soluções para: versionamento de dados de treinamento, rastreamento de parâmetros, métricas e artefatos de modelos (onde entram plataformas como MLFlow) e monitoração contínua pós-deploy (como vimos na aula anterior, monitorar data drift, performance etc. faz parte do ciclo contínuo).

Ainda assim, quando bem implementado, o CI/CD em MLOps traz benefícios enormes: reprodutibilidade (você consegue repetir todo o processo de treinamento e obter resultados consistentes, pois tudo é automatizado e versionado), velocidade de entrega (novos modelos ou updates chegam em horas ou dias, não meses), confiabilidade (menos intervenções manuais reduzem erros de implantação) e escalabilidade (pipelines podem treinar e implantar múltiplos modelos em paralelo, com orquestração adequada). Essa automação completa do workflow de ML é às vezes chamada de CT (Continuous Training) em conjunto com CI/CD, enfatizando que no ML muitas vezes queremos retreinar continuamente conforme chegam dados novos.

Para concretizar esses conceitos, vejamos a seguir as ferramentas e componentes que normalmente constituem um pipeline de CI/CD para ML.

Componentes e Ferramentas de CI/CD para ML

Em um pipeline típico de CI/CD voltado a Machine Learning, podemos identificar diversos componentes integrados:

- **Controle de versão (Git)** – armazena o código do projeto de ML (e em alguns casos metadados de dados ou modelos) e aciona pipelines quando há mudanças.
- **Serviço de Integração Contínua** – uma ferramenta que orquestra a execução automática de builds e testes quando mudanças acontecem. Exemplos: Jenkins, GitHub Actions, GitLab CI, Azure DevOps, etc.
- **Serviço de Deploy/Entrega** – gerencia a implantação dos artefatos aprovados para um ambiente (pode ser o próprio CI server realizando deploy, ou ferramentas especializadas de CD/Kubernetes).
- **Registro de Modelos** – um repositório central para versionar e gerenciar modelos de ML aprovados (ex.: MLFlow Model Registry, Amazon SageMaker Model Registry, ou mesmo registros integrados no Azure/GitLab).
- **Orquestrador de Pipelines de ML** – ferramenta para definir e executar pipelines de treinamento e processamento de dados de ML de forma escalável (ex.: Kubeflow Pipelines, Apache Airflow, Vertex AI Pipelines).
- **Feature Store** – opcionalmente, um repositório para armazenar e versionar features utilizadas pelos modelos, garantindo que treino e produção usem os mesmos cálculos (ex.: Feast, Feathr).
- **Metadados de ML** – sistemas para armazenar metadata de execuções, como parâmetros, métricas, lineage de dados, etc., trazendo maior governança (MLFlow Tracking, Kubeflow Metadata, etc.).

Nem todo projeto usará todos esses componentes, mas arquiteturas de MLOps mais maduras tendem a incorporar boa parte deles. Há diversas etapas em MLOps: desde o código/experimento inicial (Development) versionado, passando por **Continuous Integration do pipeline** (compilando componentes e executando testes), depois a **Continuous Delivery do pipeline** (implantando a pipeline atualizada em um ambiente de produção de treinamento), seguida de um **gatilho automático** que executa o pipeline de treinamento periodicamente ou sob demanda (por chegada de novos dados, por exemplo). O resultado é um modelo treinado registrado, que então passa pela etapa de **Continuous Delivery do modelo** – implantação do modelo treinado em um serviço de previsão. Finalmente, **o monitoramento** fecha o ciclo, observando a performance do modelo em produção e podendo acionar novos ciclos

de treinamento ou experimentação. Esta visão coincide com práticas de continuous training e realimentação que empresas com maturidade em MLOps aplicam.

Para implementar tudo isso, vamos conhecer as principais **ferramentas de orquestração CI/CD** e alguns **frameworks específicos de MLOps** que nos auxiliam:

- **GitHub Actions:** plataforma de CI/CD integrada ao GitHub. Permite definir workflows em arquivos YAML dentro do repositório (.github/workflows), que são executados em resposta a eventos como push, pull request, etc. Suporta runners hospedados (Ubuntu, Windows, Mac) ou auto-hospedados, e possui um marketplace com ações reutilizáveis. No contexto de ML, GitHub Actions é muito usado por ser simples de integrar – por exemplo, pode rodar testes de código, executar scripts de treinamento, e até integrar com serviços de cloud.
- **Jenkins:** ferramenta open-source clássica de CI, amplamente adotada na indústria. Jenkins é um servidor de automação que você pode hospedar, com milhares de plugins para integrar com praticamente qualquer tecnologia. Pipelines no Jenkins podem ser definidos via UI ou Pipeline as Code. Para ML, o Jenkins é utilizado principalmente em cenários que requerem mais controle do ambiente de execução ou integrações customizadas – por exemplo, executar treinamento em máquinas dedicadas ou GPUs locais. É comum em projetos on-premises e sua flexibilidade torna bastante útil, porém exige certa manutenção da infraestrutura pelo time.
- **GitLab CI/CD:** solução de integração contínua embutida na plataforma GitLab. Semelhante ao GitHub Actions no conceito de pipelines definidos em YAML (arquivo .gitlab-ci.yml), executados por runners. O GitLab CI integra-se perfeitamente com repositórios GitLab e possui recursos como pipelines multi-stage, triggers, e até um Model Registry integrado para experimentos de ML. Para equipes que já utilizam GitLab como repositório de código, ele oferece uma experiência unificada – commit do código, pipeline executando testes/treino, e deploy – tudo em um só lugar. Uma vantagem é a possibilidade de rodar pipelines tanto em runners gerenciados pela GitLab quanto por runners próprios.
- **CML (Continuous Machine Learning):** ferramenta open-source criada pela empresa Iterative focada em trazer capacidades de CI para projetos de

ML. O CML integra-se com GitHub Actions ou GitLab CI – você escreve instruções em um YAML ou use comandos do CML para, por exemplo, treinar um modelo e gerar métricas/plots, e o CML pode automaticamente gerar um relatório e comentar em um Pull Request os resultados do experimento. É uma forma de “habilitar” as ferramentas de CI comuns para ML, com recursos como comparação de métricas de modelos, tracking de datasets, etc. Por ser desenvolvida para ML, é bastante útil para incorporar boas práticas em pipelines existentes.

- **MLFlow:** apesar de não ser uma ferramenta de orquestração de CI/CD em si, o MLFlow é crucial no pipeline MLOps para rastrear experimentos, versionar modelos e até automatizar deploys. O **MLFlow Tracking** permite logar parâmetros, métricas e artefatos (como arquivos de modelo) de cada execução de treinamento; o **MLFlow Model Registry** atua como um repositório central de modelos com controle de versão, estágio (Staging, Production etc.). Integrar o MLFlow ao CI/CD significa que, a cada vez que um pipeline treina um novo modelo candidato, ele registra esse modelo no MLFlow (com um versionamento único, ligado ao código e dados usados) – isso provê linhagem de dados e experimentos completa: sabemos qual experimento (run) e quais parâmetros geraram o modelo X em produção. Além disso, o MLFlow pode servir como gatilho de deploy: por exemplo, ao marcar um modelo como "Production" no registry, um serviço de CD pode implantar automaticamente aquela versão no servidor de inferência.
- **Kubeflow Pipelines:** é uma plataforma open-source para pipelines de ML sobre Kubernetes. Diferente dos anteriores (que lidam mais com CI/CD de código), o Kubeflow atua como orquestrador das etapas de um workflow de ML (pré-processamento, treino, validação, deploy) dentro de containers em cluster. Pode-se enxergá-lo como um Airflow especializado em ML ou um complemento à infraestrutura CI: por exemplo, seu CI (Jenkins/GitHub Actions) ao invés de executar todo o treinamento, pode acionar um pipeline no Kubeflow que realiza as etapas de forma distribuída. O Kubeflow Pipelines fornece uma interface visual para acompanhar experimentos, reutiliza componentes containerizados e garante escalabilidade na nuvem.
- **Infraestrutura como Código (IaC):** no contexto de CI/CD de ML, não podemos esquecer da camada de infraestrutura. IaC significa gerenciar

servidores, recursos de nuvem, configurações de container etc. através de código (ex.: scripts do Terraform, CloudFormation, Azure ARM, etc.), em vez de configurações manuais. Para MLOps, isso é fundamental: garantir que o ambiente onde o modelo treina e serve é consistente e reproduzível. Podemos, por exemplo, usar Terraform para criar (e versionar) um cluster de Kubernetes ou instâncias de GPU necessárias, e até acionar a criação via pipelines. Adotar IaC traz benefícios como rollouts consistentes e repetíveis de ambientes de treinamento e produção, maior segurança e governança (você sabe exatamente quais recursos estão configurados e pode auditar mudanças no código), além de facilidade de escalar ou recriar ambientes em caso de falhas. Em resumo, as mesmas práticas DevOps de configuração automatizada de infra se aplicam aqui: containerização dos passos do pipeline (garantindo que o código roda igual em dev e prod, isolando dependências), uso de scripts de provisão para cloud, e gerenciamento de configuração de recursos (por exemplo, definir via código quantos pods ML de treino podem subir simultaneamente no cluster).

Naturalmente, existem outras ferramentas e serviços no ecossistema. Por exemplo, plataformas cloud como AWS oferecem **CodePipeline**/CodeBuild, e a Azure tem o **Azure DevOps Pipelines**, que também podem ser usados em MLOps. Há orquestradores específicos como **Apache Airflow** (utilizado algumas vezes para programar pipelines de dados e ML). O importante é entender que não existe solução única – muitas organizações combinam ferramentas: por exemplo, GitHub Actions para acionar testes e então um deploy que executa um pipeline Kubeflow; ou Jenkins para orquestrar tanto o treinamento quanto o deploy usando scripts de IaC. O foco deve ser nos princípios: automação e repetibilidade.

Desafios de CI/CD em ML

Vale destacar alguns desafios peculiares ao aplicar CI/CD em ML, já inferidos pelo que discutimos:

- **Versionamento de Dados e Modelos:** diferente de código (que o Git lida bem), dados de treino podem ser enormes e não cabem no repositório. Garantir que o pipeline use a versão correta do dataset para cada experimento é crítico.

- **Tempo de Execução e Recursos:** treinar modelos pode levar horas ou dias e exigir GPUs – o pipeline CI pode ser pesado. Muitas empresas optam por não treinar modelos grandes em cada commit, mas sim em jobs agendados ou quando há acúmulo de mudanças significativas. O uso de infraestrutura escalável (ex.: spot instances na nuvem) e fila de jobs faz parte do design do CI/CD de ML.
- **Testes Automatizados de Modelos:** além dos testes unitários convencionais, como testar automaticamente se um modelo “está bom”. Boas práticas incluem ter um conjunto de validação fixo e definir métricas-alvo: o pipeline pode comparar a métrica (ex.: acurácia, F1-Score) do novo modelo com a do modelo atual em produção. Se não houver ganho ou se houver decréscimo significativo, talvez seja válido barrar a promoção automática. Testes de sanity também podem ser codificados (ex.: garantir que o output médio não diverge muito ou que não aparece NaN durante o treino).
- **Reprodutibilidade:** pequenas diferenças de ambiente quebram a reprodutibilidade. Por isso, containerização (Docker) e gerenciamento de dependências determinísticas (pin de versões em requirements.txt ou environment.yml) são obrigatórios. Um desafio é que mesmo com container, a não ser que se fixe seeds aleatórios, os modelos podem ter pequenas variações de run para run. Então, para fins de CI, é útil fixar seeds nos experimentos de teste para resultados comparáveis.
- **Segurança e Governança:** automatizar deploys exige confiança no processo. Precisamos ter certeza de que se vamos lançar um modelo direto em produção, ele passou por todas as validações. Em cenários críticos (saúde, finanças), costuma-se manter aprovação humana como etapa final. Além disso, pipelines precisam lidar com segredos (credenciais de banco de dados, chaves de API) de forma segura – as plataformas de CI oferecem armazenamento de secrets para isso.
- **Integração com Monitoramento:** fechar o ciclo CI/CD/CM (Continuous Monitoring) – idealmente o próprio pipeline pode ser acionado por alertas de monitoramento (por exemplo, se o monitoramento detectar data drift, ele aciona uma execução do pipeline de retraining). Essa integração ainda é

uma fronteira para muitas empresas, mas faz parte da visão de MLOps avançado.

Vamos nos aprofundar agora em algumas práticas e ferramentas avançadas relacionadas a CI/CD e automação de pipelines em ML, complementando o que já vimos.

Pipelines Orquestrados com Kubeflow e TFX

No hands on usamos um pipeline bem direto dentro do CI. Em projetos de maior porte, especialmente com grandes volumes de dados, é comum separar a orquestração do workflow de ML em sistemas dedicados, como o Kubeflow Pipelines. O Kubeflow permite definir visualmente (ou via código Python DSL) um DAG (Directed Acyclic Graph) de etapas de ML – por exemplo: coleta de dados -> pré-processamento -> treino -> validação -> deploy. Cada etapa roda em um container isolado, podendo escalar independentemente. A integração com CI/CD ocorre no gatilho: em vez de rodar `python train.py` diretamente no CI, podemos configurar para que um commit no Git dispare a execução de um pipeline Kubeflow (que já executa todas etapas). Como fazer isso? Uma abordagem é usar a própria API do Kubeflow Pipelines ou scripts `kubectl` no CI para subir uma Run nova no Kubeflow.

Uma vantagem dessa separação é que o Kubeflow provê um ambiente otimizado para workloads de ML, incluindo gestão de artefatos, experimentos e comparações de runs, cache de etapas (se os dados não mudaram, não reexecuta certas partes) e fácil integração de componentes (ex.: um componente de data validation ou de bias detection pode ser plugado no pipeline).

Além do Kubeflow, merece menção o TensorFlow Extended (TFX) – um framework da Google que define pipelines de ML de ponta a ponta, com componentes para ingestão de dados, transformação, treino, validação de modelo (TensorFlow Model Analysis) e deploy. O TFX pode rodar pipelines em orquestradores como Airflow, Kubeflow ou Beam. É especialmente útil se seu stack é baseado em TensorFlow, mas muitos conceitos dele (como o Validator de dados ou a validação de modelo) podem ser adotados independentemente. Um pipeline TFX bem montado integrado ao CI vai assegurar que, por exemplo, nenhum modelo vai a prod sem passar por verificações de drift, skew entre treino e serviço etc. Isso eleva o nível de confiança no CI/CD.

Infrastructure as Code e ambientes reprodutíveis

Retomando Infraestrutura como Código (IaC), vamos detalhar como se aplica diretamente em MLOps: imagine que seu pipeline de retraining precisa de um ambiente com certas características – por exemplo, uma máquina virtual com GPU e 16 GB de RAM para processar dados. Com IaC, você pode codificar essa infraestrutura. Uma ferramenta popular é o Terraform, que permite escrever arquivos declarando recursos (VMs, containers, rede etc.). No contexto CI/CD, podemos ter uma etapa no pipeline que executa terraform apply antes de iniciar o treinamento, garantindo que a infra necessária está de pé. Isso pode incluir criar uma instância de computação, configurar variáveis ou mesmo provisionar serviços de banco de dados para buscar dados.

Um cenário prático: seu pipeline mensal de treinamento processa milhões de registros. Em vez de manter um servidor ligado o tempo todo, você pode usar IaC no CI/CD para on demand, subir instâncias de processamento, executar o job de ML nelas e depois destruí-las – otimizando custos. Ferramentas de IaC como Terraform e Pulumi podem ser integradas no pipeline com relativa facilidade.

Além disso, a IaC contribui para reprodutibilidade e padronização. Ao gerenciar infra via código, garantir que dev, staging e prod são consistentes fica muito mais fácil (é só aplicar os mesmos módulos com configurações diferentes). Em outras palavras, a infraestrutura deixa de ser um obstáculo misterioso e vira parte do pipeline versionado – se alguém alterar a configuração de um cluster, isso passa por código (pull request, revisão etc.), alinhado ao espírito de DevOps.

Uma ferramenta relacionada é o uso de Docker/Containers. Embora não seja IaC no sentido estrito, containerizar aplicações de ML (por exemplo, criar uma imagem Docker com todo o ambiente necessário para treinar ou servir o modelo) é uma forma de garantir que o que rodamos no CI é igual ao que rodamos local ou em produção. No CI, podemos buildar imagens automaticamente e até versioná-las com o commit hash. Kubernetes, por sua vez, nos permite implantar essas imagens de forma escalável. Tudo isso compondo a infraestrutura tratada como código/configuração, não passos manuais.

Validação Automatizada de Dados e Modelos

Uma prática de engenharia de software que ganha cada vez mais espaço em ML é incluir validação automática de dados e checagens de qualidade de modelo como parte da pipeline. Ferramentas como Great Expectations, Pandera e Deepchecks, entre outras, atuam como “testes unitários” para os dados. Você pode, por exemplo, definir expectativas: “coluna X não deve ter nulos, coluna Y (idade) deve estar entre 0 e 120, distribuição de categoria Z não deve divergir mais que 10% do histórico”. Essas regras, quando incorporadas no pipeline de CI, impedem que um novo dado sujo degrade seu modelo silenciosamente.

Considere um exemplo: um modelo de crédito espera que a variável “salário” seja sempre positiva. Se por algum bug começarem a entrar valores negativos, um step de validação pode pegar isso e abortar o processo antes mesmo de treinar o modelo errado. Essas garantias podem salvar uma empresa de um modelo mal treinado para a produção.

De forma similar, validação de modelo pode incluir: checar se o modelo novo supera ou ao menos iguala o antigo em certo conjunto de teste; validar se não surgiram viés indesejados (por exemplo, adicionar um teste que verifica diferença de performance do modelo entre subgrupos demográficos – algo que pode ser automatizado até certo ponto).

Claro, nem tudo pode ser automatizado – muitas questões de fairness ou de validação complexa requerem análise humana – mas quanto mais pudermos usar checagens automáticas objetivas, mais robusto fica o processo.

MERCADO, CASES E TENDÊNCIAS

A automação de pipelines e MLOps como um todo evoluiu rapidamente nos últimos anos, tornando-se praticamente obrigatórios em empresas que escalam modelos em produção. Vamos discutir algumas tendências de mercado, exemplos de adoção e perspectivas futuras:

- **Adoção Massiva de MLOps:** pesquisas indicam que o MLOps já está se tornando mainstream nas organizações. Em 2022, cerca de 85% das empresas já possuíam um orçamento dedicado a iniciativas de MLOps, reconhecendo a importância de operacionalizar modelos de ML de forma eficiente. Além disso, em 2023, praticamente 98% das empresas planejavam aumentar investimentos em MLOps, mais da metade planejando ampliar gastos em mais de 25%. Esses números mostram um forte comprometimento da indústria em aprimorar pipelines de ML, indo muito além de projetos ad-hoc de ciência de dados. O mercado de ferramentas e plataformas de MLOps reflete isso – as projeções estimam que o mercado global de MLOps cresça de ~3,6 bilhões de dólares em 2025 para quase 8,7 bilhões até 2033, um crescimento bem acima da média do setor de TI.
- **Demandas por velocidade e eficiência:** por que esse investimento todo? As empresas perceberam que ter modelos de IA é bom, mas manter esses modelos atualizados e com qualidade em produção é desafiador sem automação. Com a rápida mudança de dados e de negócios, quem conseguir treinar e implantar melhorias de modelos continuamente leva vantagem competitiva. Por exemplo, no setor de e-commerce, modelos de recomendação precisam ser atualizados com novos hábitos de consumo; no financeiro, modelos antifraude devem incorporar rapidamente as novas táticas de fraudadores. Sem pipelines automatizados, isso levaria meses; com MLOps, em dias ou horas um novo modelo vai ao ar assim que há evidências de degradação do antigo.
- **Cases de Sucesso:** diversas organizações líderes construíram infraestruturas robustas de CI/CD para ML. A Uber, por exemplo,

desenvolveu internamente a plataforma Michelangelo, que automatiza desde o preparo de dados até o deploy e monitoramento de modelos, atendendo centenas de casos de uso de ML dentro da empresa. A Netflix investiu em pipelines para suas recomendações e sistemas de conteúdo – embora com menos detalhes públicos, sabe-se que praticam experimentação contínua (A/B) de modelos em produção.

- **Integração com DevOps e plataformas de Engenharia de Dados:** uma tendência forte é a convergência de MLOps com as práticas de DevOps e DataOps tradicionais. Ferramentas de CI/CD como Jenkins, GitHub Actions etc., estão adicionando recursos ou integrações voltadas a ML. Ao mesmo tempo, plataformas de dados (Databricks, Dataiku etc.) incorporam funcionalidades de pipeline automation e deploy de modelos. Vemos surgir o papel de Platform Engineer focado em ML – profissionais que constroem essas plataformas internas para cientistas de dados usarem sem precisar reinventar. Em termos de cultura, times de desenvolvimento e de dados estão colaborando de maneira mais próxima, muitas vezes usando os mesmos repositórios e pipelines para código de aplicação e código de modelos (especialmente em aplicações de aprendizado de máquina embarcado em produtos).
- **Automatização + Human-in-the-loop:** apesar de falarmos muito de automação completa, outro movimento no mercado é Human-in-the-loop MLOps, onde automação e intervenção humana coexistem para melhores resultados. Por exemplo, uma empresa pode automatizar 90% do pipeline, mas ter um(a) especialista revisando o modelo antes do deploy final ou aprovando gates de qualidade é fundamental para garantir qualidade. Ferramentas de CI/CD estão adicionando recursos de aprovação, relatórios ricos (ex.: gerar automaticamente um report de performance do modelo para o revisor). Isso é uma tendência especialmente em setores regulamentados que exigem accountability – o modelo só vai pra produção se um humano aprovar, mas todo o trabalho braçal até chegar a esse ponto foi automatizado.
- **Tendências Tecnológicas:** algumas tendências tecnológicas em MLOps pipelines incluem: uso de CI/CD declarativo (ex.: pipelines declarados em

YAML, tanto para treinamento quanto deploy, facilitando reuso e padronização – Kubeflow e Airflow já seguem essa linha, inclusive com pipelines as code); serverless ML pipelines – utilizando serviços gerenciados onde possível (Cloud Functions para pequenas tarefas etc., reduzindo overhead de infra); e crescimento de Feature Stores integrados – garantindo que o pipeline CI/CD trate features como um artefato versionado. Outra tendência recente é adaptar CI/CD para modelos de Deep Learning e Large Language Models, que têm particularidades (treinos caríssimos, necessidade de gerenciamento de checkpoints etc.). Já se fala em Continuous Retraining para LLMs com dados atualizados. Ferramentas open-source como Hugging Face Hub começam a oferecer integrações de CI para validar e executar testes em modelos de NLP de forma automatizada quando publicados.

- **Desafios na Adoção:** apesar dos benefícios, empresas encontram obstáculos na adoção de MLOps. Falta de mão de obra qualificada é um ponto geralmente citado – equipes de data science nem sempre têm alguém experiente em DevOps/MLOps para configurar esses pipelines. Isto tem levado a uma procura alta por MLOps Engineers no mercado. Além disso, as organizações estão investindo em treinamento interno para que cientistas de dados adquiram noções de CI/CD. A cultura também precisa mudar: trabalho colaborativo, code review de notebooks convertidos em scripts etc., são novidades para muitos. Porém, a direção é clara: cada vez mais, data scientists precisam pelo menos entender o básico de MLOps, enquanto engenheiros(as) de software aprendem particularidades de ML – essa interseção de habilidades é o que define a prática de MLOps.
- **ROI e Qualidade:** empresas que já implementaram CI/CD para seus fluxos de ML reportam ganhos notáveis: reduções no tempo de deploy de modelos (de semanas para dias ou menos), aumento da frequência de atualização (modelos melhorados continuamente em vez de ficarem obsoletos) e melhora na confiança/regulamentação. Por exemplo, uma companhia de saúde que automatizou pipelines conseguiu rastrear precisamente qual versão do modelo foi usada em cada previsão clínica – fundamental para auditorias e conformidade regulatória. Outra do ramo de manufatura reduziu

desperdício detectando drifts de modelo em horas (via monitoramento acionando pipelines) em vez de descobrir meses depois quando perdas já haviam ocorrido. Esses casos reforçam que CI/CD não é “luxo”, e sim necessidade para garantir qualidade consistente e valor contínuo de sistemas de IA.

Em suma, o mercado caminha para uma adoção ampla de CI/CD e automação em Machine Learning. Estamos vendo a consolidação de best practices, o surgimento de ferramentas cada vez mais amigáveis (muitas vezes abstraindo a complexidade – ex.: já há ferramentas low-code para montar pipelines de ML visuais que por baixo fazem CI/CD). Para quem se especializa nisso, as oportunidades são vastas: desde construir plataformas internas, até atuar em empresas SaaS que fornecem soluções MLOps. A tendência é de crescimento e amadurecimento, com MLOps tornando-se tão comum quanto DevOps é hoje. E como resultado, devemos ter modelos de ML mais confiáveis, atualizados e integrados aos produtos e serviços de forma transparente.

O QUE VOCÊ VIU NESTA AULA?

A aula focou na contextualização do desafio da manutenção contínua de modelos que estão em ambiente de produção e como o processo de CI/CD adaptado à ML (MLOps) ajuda a enfrentá-lo, automatizando o re-treinamento e o deployment de modelos de forma reprodutível, viabilizando a atualização constante por pipelines automatizados. Em seguida, foi apresentada a definição de CI (Continuous Integration) e CD (Continuous Delivery/Deployment) no contexto de Machine Learning, incluindo diferenças em relação ao DevOps tradicional.

Além disso, foram apresentadas diversas ferramentas e plataformas relevantes, incluindo serviços de CI/CD como GitHub Actions, GitLab CI e Jenkins, cada um com seu caso de uso; ferramentas focadas em MLOps como MLFlow (para experiment tracking e model registry) e Kubeflow (para orquestração de pipelines de ML escaláveis); e conceitos como Infrastructure as Code (Terraform, etc.) integrados ao pipeline para garantir ambientes consistentes.

A parte prática consistiu em um hands on configurando um pipeline de CI/CD usando GitHub Actions e MLFlow. Neste exercício, foi definido um workflow YAML que executa testes e treino automaticamente a cada commit, usando secrets para se conectar a um servidor MLFlow e logar parâmetros, métricas e artefatos do modelo treinado. Foi mostrado um exemplo de código Python integrando MLFlow e discutiu-se como esse pipeline pode ser estendido para deploy contínuo, ilustrando na prática os conceitos teóricos e mostrando como implementar rastreabilidade e automação ponta-a-ponta, do commit ao modelo registrado.

Por fim, foram exploradas práticas recomendadas de engenharia de software aplicada a ML, como validação automatizada de dados (para prevenir falhas), testes automatizados de comportamento de modelos (ex.: comparar métricas com baseline), uso de ambientes isolados e containerizados para garantir reprodutibilidade e monitoração contínua integrando com pipelines. Tudo isso contribui para fluxos de ML mais confiáveis.

Com isso, concluímos que o uso de CI/CD e automação de pipelines é parte fundamental do ciclo de vida de modelos modernos. A combinação de cultura (práticas de DevOps), processos (pipelines bem definidos) e ferramentas adequadas permite

escalar o uso de inteligência artificial de forma segura e eficaz, garantindo que modelos continuem entregando valor ao longo do tempo.

EMSE

REFERÊNCIAS

CLEARML BLOG. **MLOps in 2023: What Does the Future Hold?** 2023. Disponível em: <https://clear.ml/blog/mlops-in-2023>. Acesso em: 28 jan. 2026.

GITHUB ACTIONS DOCUMENTATION. **Understanding GitHub Actions**. 2025. Disponível em: <https://docs.github.com/en/actions/get-started/understand-github-actions>. Acesso em: 28 jan. 2026.

GOOGLE CLOUD ARCHITECTURE CENTER. **MLOps: Continuous delivery and automation pipelines in machine learning**. 2024. Disponível em: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>. Acesso em: 28 jan. 2026.

LOGICMONITOR BLOG. **Ops Explained: AIOps vs. DevOps vs. MLOps vs. Agentic AIOps**. 2025. Disponível em: <https://www.logicmonitor.com/blog/aiops-devops-mlops-and-agentic-aiops>. Acesso em: 28 jan. 2026.

MLOPS GUIDE. **CI/CD for Machine Learning**. 2021. Disponível em: <https://mlops-guide.github.io>. Acesso em: 28 jan. 2026.

MLFLOW DOCUMENTATION. **MLflow Model Registry**. s.d. Disponível em: <https://mlflow.org>. Acesso em: 28 jan. 2026.

OJEABULU, G. **Stop ML Model Failures: Complete Guide to Data Validation**. 2025. Disponível em: <https://medium.com/data-science-collective/stop-ml-model-failures-complete-guide-to-data-validation-with-pandera-great-expectations-dbt-d7656eeadfae>. Acesso em: 28 jan. 2026.

RED HAT DEVELOPER. **Implement MLOps with Kubeflow Pipelines**. 2024. Disponível em: <https://developers.redhat.com/articles/2024/01/25/implement-mlops-kubeflow-pipelines>. Acesso em: 28 jan. 2026.

RYABOKON, D. **CI/CD/CT with MLflow and GitHub Actions**. 2023. Disponível em: <https://medium.com/@d.ryabokon/ci-cd-ct-with-mlflow-and-github-actions-2a6ba5c767dd>. Acesso em: 28 jan. 2026.

STRAITS RESEARCH. **MLOps Market Size, Share, Trends & Growth by 2033**. 2024. Disponível em: <https://straitsresearch.com/report/mlops-market>. Acesso em: 28 jan. 2026.

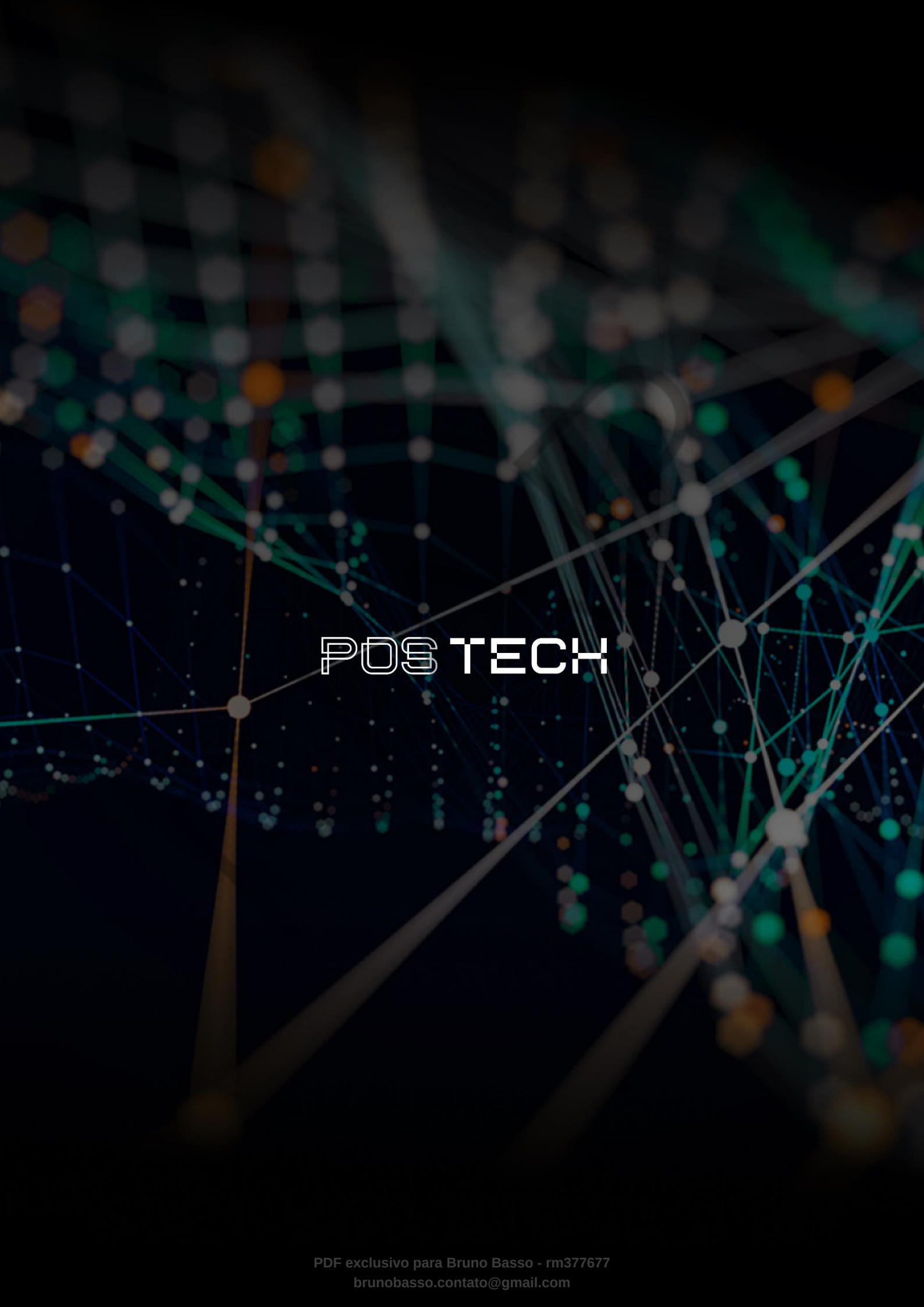
VAISHNAV. **ML CI/CD Pipeline: Architecture, Tools, and Real-World Structure**. 2025. Disponível em: <https://medium.com/@vaishnav0501/ml-ci-cd-pipeline-architecture-tools-and-real-world-structure-d10233d601c9>. Acesso em: 28 jan. 2026.

ZENML BLOG. **Infrastructure-as-Code (IaC) for MLOps with Terraform & ZenML**. 2024. Disponível em: <https://www.zenml.io/blog/mlops-terraform-zenml>. Acesso em: 28 jan. 2026.

PALAVRAS-CHAVE

CI/CD. MLOps. Automação.

EMENDAS



POSTECH